

POLITECNICO
MILANO 1863

# Bioinformatics Algorithms

## Sequences and sequence alignment (part 2)

Rosario M. Piro
(based on slides by Giulio Pavesi)

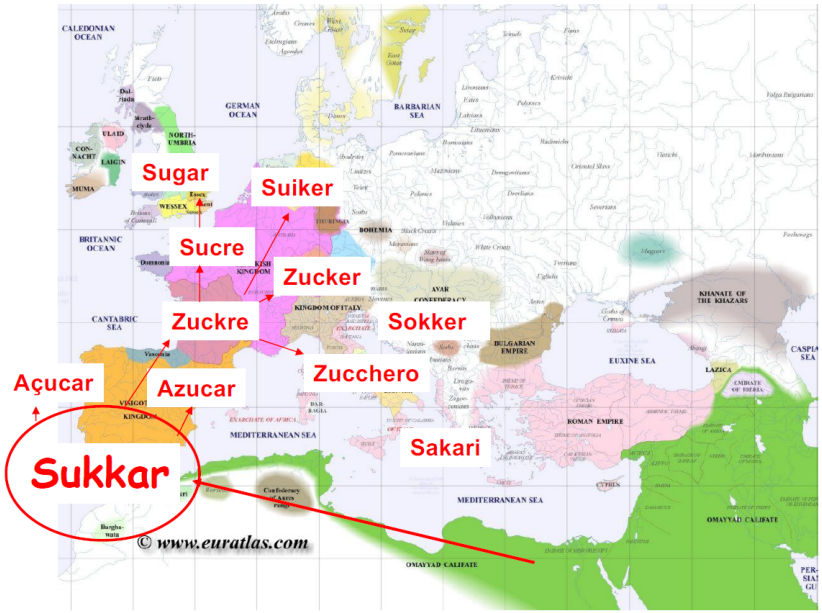
Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB)
Politecnico di Milano

# Comparing words

- If two words are similar, we can often infer the meaning of one from the meaning of the other
  - Usually, two words are similar when they share a common history in the evolution of languages

- Example: *a*="SUGAR" and *b*="SUCRE"

```
SUGAR     SUGAR-     SUGAR-
SUCRE     SUC-RE     SU-CRE
```



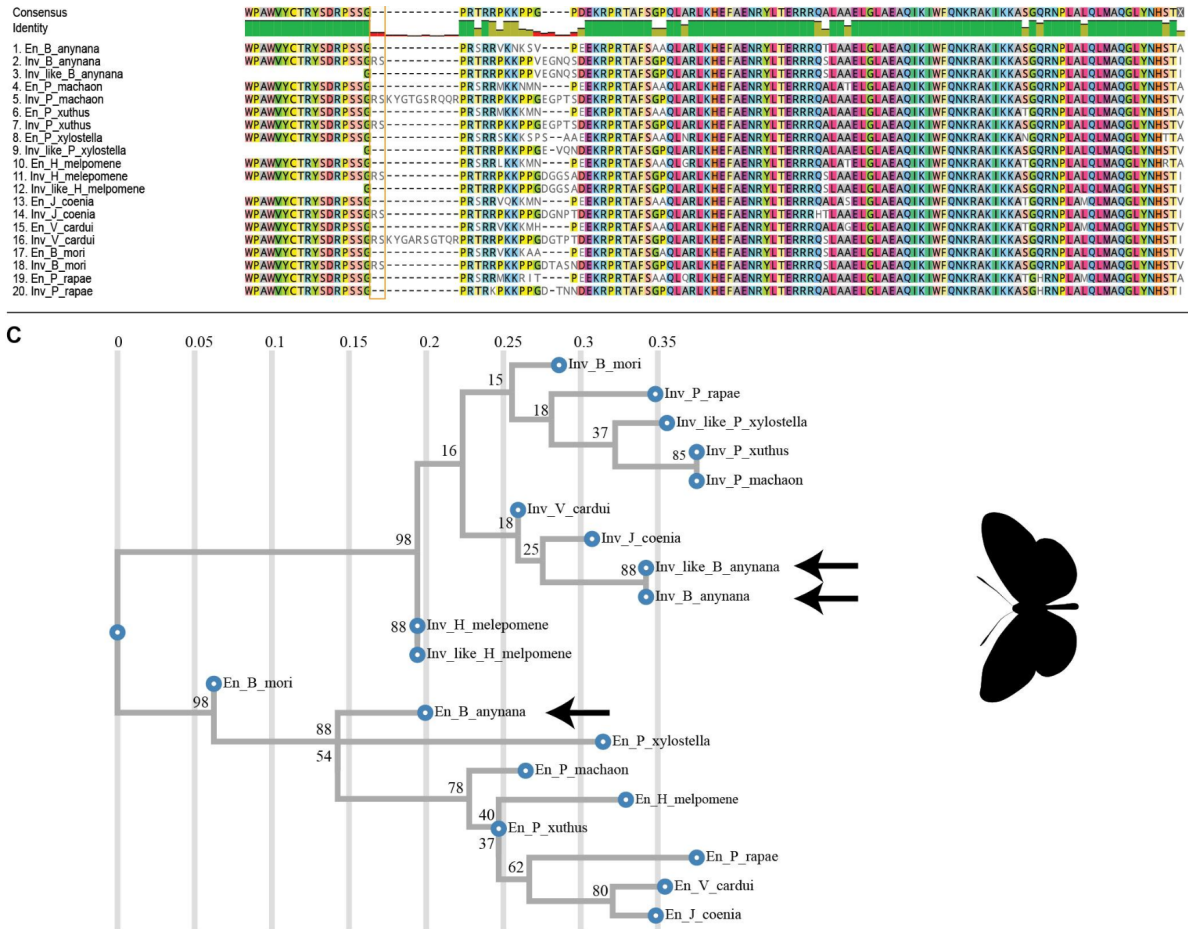

Europe, AD 700

- The words denoting "sugar" of all modern European languages are descendants of a common ancestral word ("sukkar"; arabic)

- For some of the words, we also know "closer" (more recent) ancestors; e.g., the medieval French word "zuckre" (itself a descendant of "sukkar") is the ancestor of modern French "sucre", Italian "zucchero", German "Zucker" and others

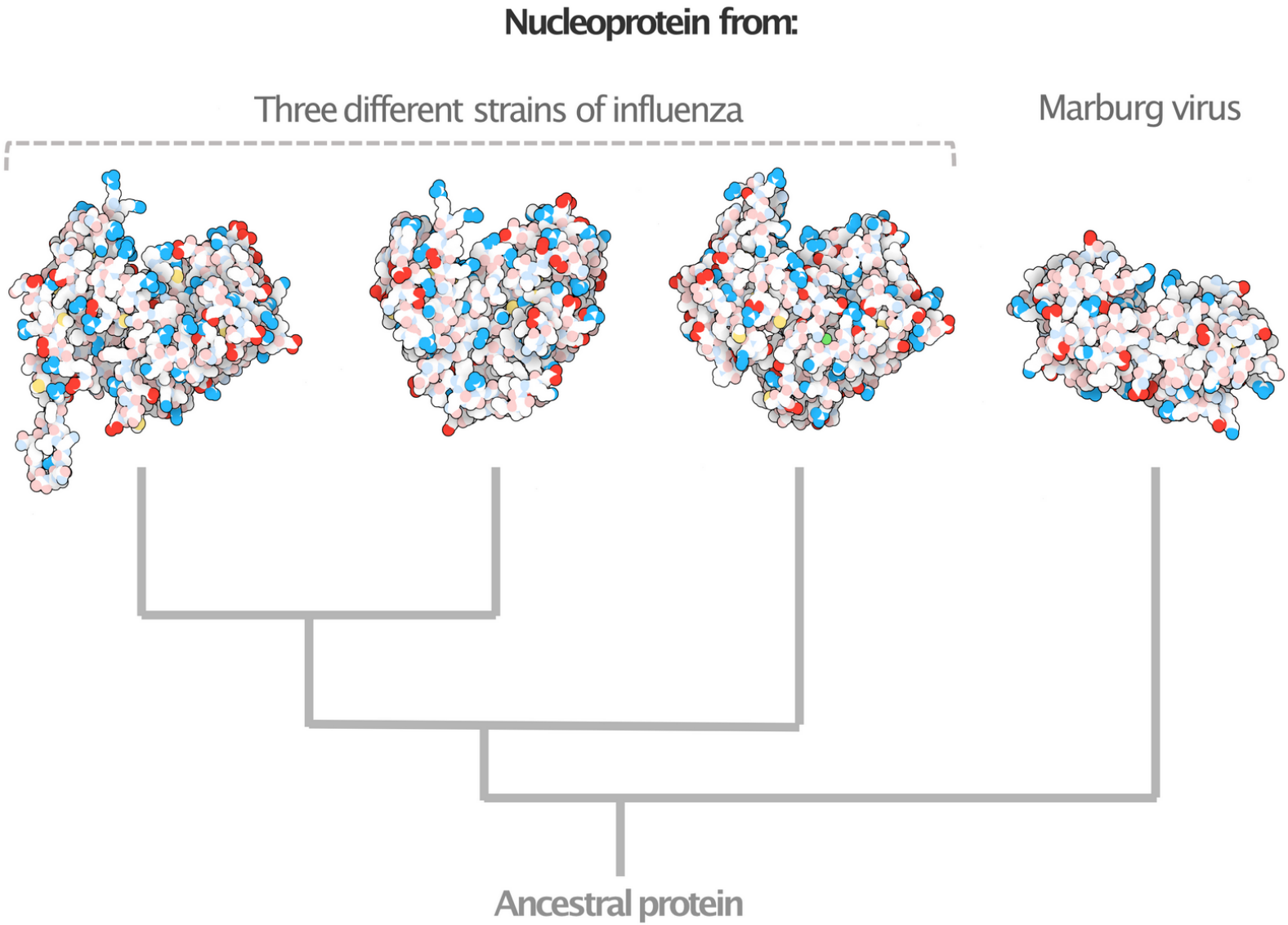
# Comparing biological sequences

- If two sequences are similar, we can often infer the function of one from the biological function of the other
  - Usually, two sequences are similar when they share a common evolutionary history (close in the evolutionary tree)



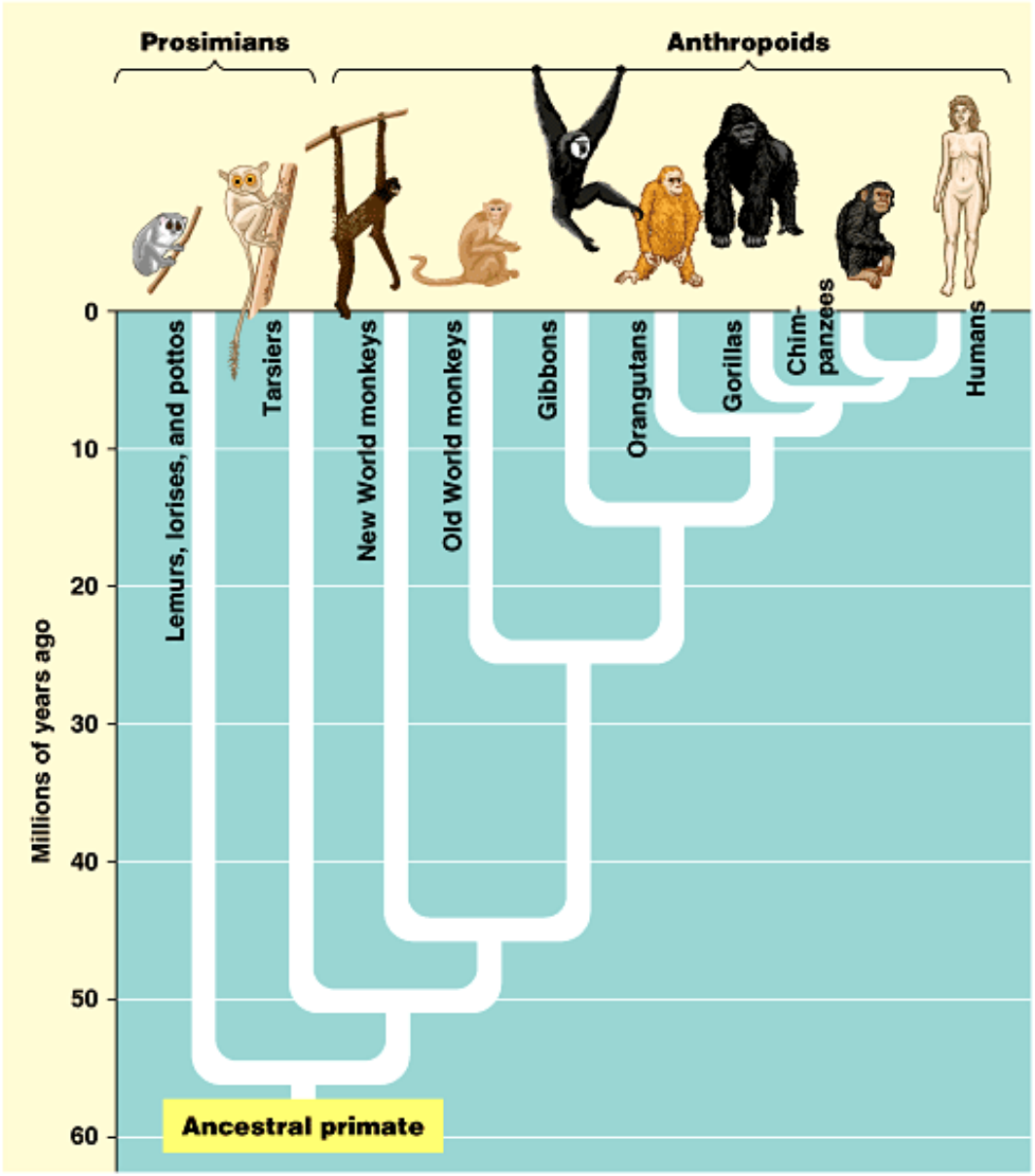

(from Banerjee et al., 2020)

# Protein evolution



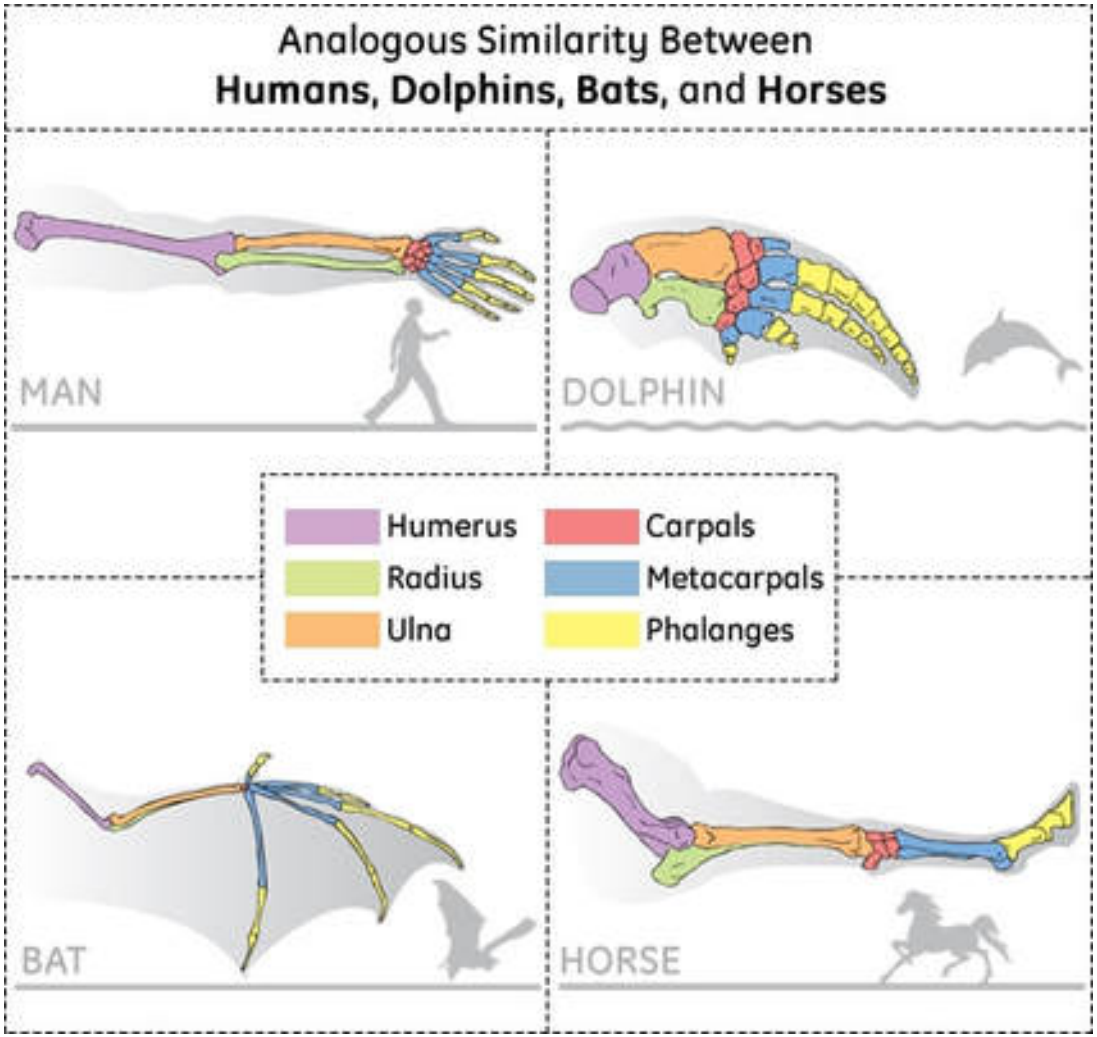

(from https://sitn.hms.harvard.edu/flash/2020/what-can-evolution-teach-us-about-the-viruses-of-the-future/)

# Primate evolution



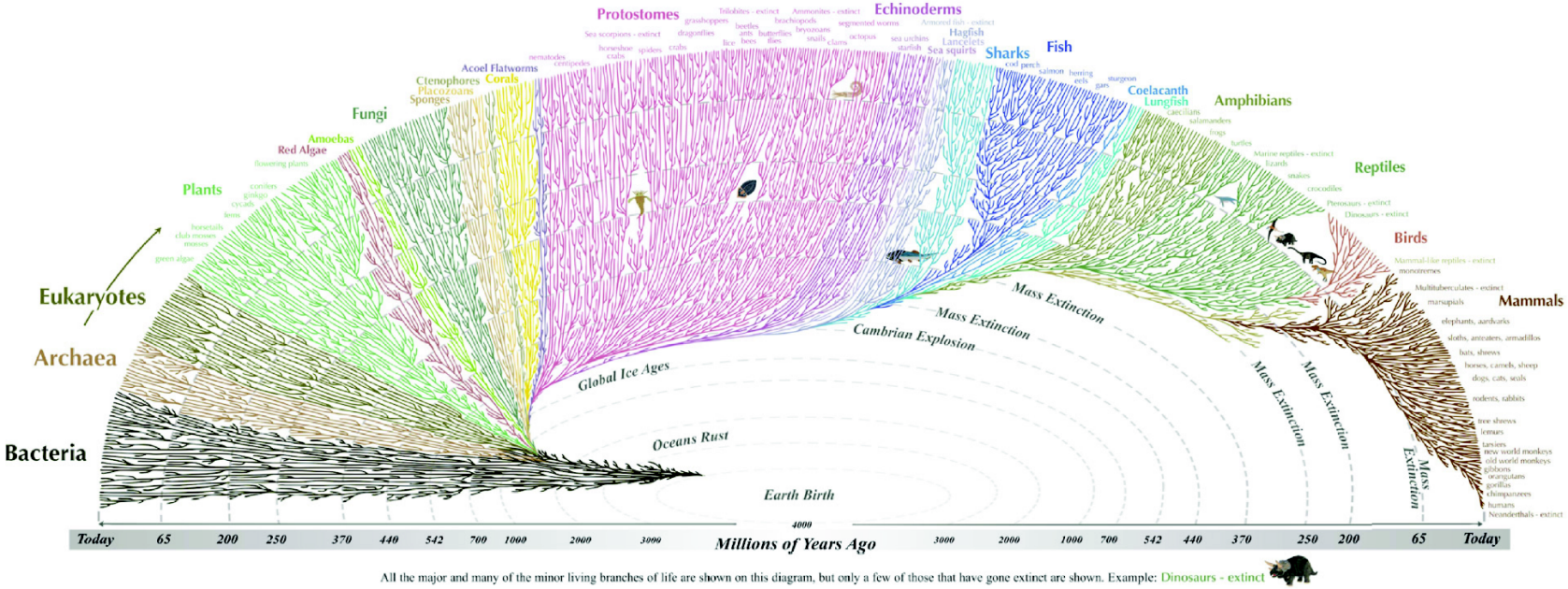

Copyright © Pearson Education, Inc., publishing as Benjamin Cummings.

# Limb evolution




(from BiologyWise)

# Tree of life




(from: `https://www.evogeneao.com/en/explore/tree-of-life-explorer`; see there for an interactive version!)

- Hypothesis: all species have a common evolutionary history (common origin!)
  - During evolution, new species are derived from existing ones (“speciation”)
  - If so, we should be able to find traces of evolution within the DNA; the DNA of two species should be somewhat similar (same for genes and proteins they encode)!
  - The more recent the divergence, the more similar the DNA/genes/proteins should be
  - This explains why we share some essential genes and proteins even with bacteria ...
  - By comparing sequences we should be able to reconstruct the “genomic history” of the species we can observe today
  - Powerful tool for comparing sequences: the alignment!

# Example: protein sequences

Alignment example, split over multiple lines:

```
SSH_UOMO        -MLLLARCLLLVLVSSLLVCSGLACGPGRGFGKRRHPKKLTPLAYKQFIPNVAEKTLGAS
SSH_TOPO        MLLLLARCFLVILASSLLVCPGLACGPGRGFGKRRHPKKLTPLAYKQFIPNVAEKTLGAS
                 :******:*::*.******.************************************

SSH_UOMO        GRYEGKISRNSERFKELTPNYNPDIIFKDEENTGADRLMTQRCKDKLNALAISVMNQWPG
SSH_TOPO        GRYEGKITRNSERFKELTPNYNPDIIFKDEENTGADRLMTQRCKDKLNALAISVMNQWPG
                *******:****************************************************

SSH_UOMO        VKLRVTEGWDEDGHHSEESLHYEGRAVDITTSDRDRSKYGMLARLAVEAGFDWVYYESKA
SSH_TOPO        VKLRVTEGWDEDGHHSEESLHYEGRAVDITTSDRDRSKYGMLARLAVEAGFDWVYYESKA
                ************************************************************

SSH_UOMO        HIHCSVKAENSVAAKSGGCFPGSATVHLEQGGTKLVKDLSPGDRVLAADDQGRLLYSDFL
SSH_TOPO        HIHCSVKAENSVAAKSGGCFPGSATVHLEQGGTKLVKDLRPGDRVLAADDQGRLLYSDFL
                ************************************ ********************

SSH_UOMO        TFLDRDDGAKKVFYVIETREPRERLLLTAAHLLFVAPHNDSATGEPEASSGSGPPSGGAL
SSH_TOPO        TFLDRDEGAKKVFYVIETLEPRERLLLTAAHLLFVAPHND----------SGPTPG---
                ******:*********** ********************          ***..*

SSH_UOMO        GPRALFASRVRPGQRVYVVAERDGDRRLLPAAVHSVTLSEEAAGAYAPLTAQGTILINRV
SSH_TOPO        -PSALFASRVRPGQRVYVVAERGGDRRLLPAAVHSVTLREEEAGAYAPLTAHGTILINRV
                 * ******************.*************** ** *********:********

SSH_UOMO        LASCYAVIEEHSWAHRAFAPFRLAHALLAALAPARTDRGGDSGGGDRGGGGGRVALTAPG
SSH_TOPO        LASCYAVIEEHSWAHRAFAPFRLAHALLAALAPARTD----------GGGGGSIP-AAQS
                ************************************          ***** :. :* .

SSH_UOMO        AADAPGAGATAGIHWYSQLLYQIGTWLLDSEALHPLGMAVKSS
SSH_TOPO        ATEARGAEPTAGIHWYSQLLYHIGTWLLDSETMHPLGMAVKSS
                *::* ** .************:*********::**********
```

Proteins encoded by the human *SHH* gene (top) and the mouse *SHH* gene (bottom)

SSH = sonic hedgehog signaling molecule

# Example: protein sequences

Another alignment example, split over multiple lines:

**Range 1: 1 to 136** GenPept Graphics ▼ Next Match ▲ Previous Match

| Score | Expect | Method | Identities | Positives | Gaps |
|---|---|---|---|---|---|
| 248 bits(633) | 3e-87 | Compositional matrix adjust. | 121/136(89%) | 130/136(95%) | 0/136(0%) |

```
Query   1     MARTKQTARKSTGGKAPRKQLATKAARKSAPATGGVKKPHRYRPGTVALREIRRYQKSTE   60
              MARTKQTARKSTGGKAPRKQLA+KAARKSAP+TGGVKKPHRY+PGTVALREIRR+QKSTE
Sbjct   1     MARTKQTARKSTGGKAPRKQLASKAARKSAPSTGGVKKPHRYKPGTVALREIRRFQKSTE   60

Query   61    LLIRKLPFQRLVREIAQDFKTDLRFQSSAVMALQEACEAYLVGLFEDTNLCAIHAKRVTI   120
              LLIRKLPFQRLVREIAQDFKTDLRFQSSA+ ALQE+ EAYLV LFEDTNL AIHAKRVTI
Sbjct   61    LLIRKLPFQRLVREIAQDFKTDLRFQSSAIGALQESVEAYLVSLFEDTNLAAIHAKRVTI   120

Query   121   MPKDIQLARRIRGERA  136
                KDI+LARR+RGER+
Sbjct   121   QKKDIKLARRLRGERS  136
```

Proteins encoded by the human histone 3 (*H3*) gene (top) and the homologuous gene of *S. cerevisiae* (yeast; bottom)

# Alignment recap

- Let $a$ and $b$ be two sequences we want to compare, with $|a| = n$ and $|b| = m$

- We use a measure which considers both similarities (equal characters) and differences (substitutions, insertions, deletions)
  - Similarities: positive score
  - Differences: negative score

```
  C   A   A   G   A   C
  G   A   A   -   -   C
-1  +1  +1  -2  -2  +1  = -2
```

  - Recursive computation of alignment scores

$$A\big(a(i), b(j)\big) = \max \begin{cases} \mathbf{w_d} + A\big(a(i), b(j-1)\big) \\ \mathbf{w_d} + A\big(a(i-1), b(j)\big) \\ \begin{cases} \mathbf{w_m} + A\big(a(i-1), b(j-1)\big) & \text{if } a_i = b_j \\ \mathbf{w_s} + A\big(a(i-1), b(j-1)\big) & \text{if } a_i \neq b_j \end{cases} \end{cases}$$

  - $\mathbf{w_d} < 0$: negative **gap penalty** for insertions/deletions
  - $\mathbf{w_s} < 0$: negative **mismatch penalty** for substitutions
  - $\mathbf{w_m} > 0$: positive **match score** for the same symbol

# Substitution matrix

- We can actually simplify this when using a substitution matrix, denoted by $\sigma$:
  - Given an alphabet $\Sigma$, the substitution matrix is a $|\Sigma| x |\Sigma|$-matrix which gives for every pair of symbols $s_i$, $s_j \in \Sigma$ the "weight" of the substituting $s_i$ by $s_j$ in an alignment
  - The matrix is symmetrical, i.e., $\sigma(s_i, s_j) = \sigma(s_j, s_i)$
    - ★ Example for nucleotides: $\sigma(\mathtt{C},\mathtt{T}) = \sigma(\mathtt{T},\mathtt{C})$
  - Given a specific symbol $s_i$, the values $\sigma(s_i, s_j)$ can be different for the possible substitutions by different $s_j$
    - ★ Example for amino acids: $\sigma(\mathrm{Ala}, \mathrm{Cys})$ may be different from $\sigma(\mathrm{Ala}, \mathrm{Ser})$
  - For $i = j$ this matrix can also specify the "weight" of substituting $s_i$ by itself, which is equivalent to the positive score associated with a match of the symbol in the alignment!
    - ★ Examples: $\sigma(\mathtt{C},\mathtt{C})$; $\sigma(\mathrm{Ala}, \mathrm{Ala})$
  - The positive scores for "matches", i.e., $\sigma(s_i, s_i)$ can differ from symbol to symbol!
    - ★ Example: $\sigma(\mathrm{Ala}, \mathrm{Ala})$ may be different from $\sigma(\mathrm{Cys}, \mathrm{Cys})$

# Substitution matrix

The simplest substitution matrices are those for DNA/RNA sequences

- Postive scores for all matches are the same
  All matches are equally positive!
- Negative scores for all mismatches/substitutions are the same
  All mismatches are equally penalized!

- Example:

| | A | C | G | T |
|---|---|---|---|---|
| A | +2 | -1 | -1 | -1 |
| C | -1 | +2 | -1 | -1 |
| G | -1 | -1 | +2 | -1 |
| T | -1 | -1 | -1 | +2 |

# Substitution matrix

For amino acid sequences, we usually use more complex substitution matrices

- Not all matches are equally important!
- Different mismatches/substitutions have different penalization!
  - Some may have zero or even low positive weights
    - ★ "Conservative" substitutions (weight $\geq 0$)
    - ★ "Quasi matches" (weight $> 0$); assumed to not affect protein structure/function; More often found in evolution; e.g., glutamate (Glu) $\leftrightarrow$ glutamine (Gln)
- Example:

| | Ala | Arg | Asn | Asp | Cys | Gln | Glu | Gly | His | Ile | Leu | Lys | Met | Phe | Pro | Ser | Thr | Trp | Tyr | Val |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Ala** | 4 | | | | | | | | | | | | | | | | | | | |
| **Arg** | −1 | 5 | | | | | | | | | | | | | | | | | | |
| **Asn** | −2 | 0 | 6 | | | | | | | | | | | | | | | | | |
| **Asp** | −2 | −2 | 1 | 6 | | | | | | | | | | | | | | | | |
| **Cys** | 0 | −3 | −3 | −3 | 9 | | | | | | | | | | | | | | | |
| **Gln** | −1 | 1 | 0 | 0 | −3 | 5 | | | | | | | | | | | | | | |
| **Glu** | −1 | 0 | 0 | 2 | −4 | 2 | 5 | | | | | | | | | | | | | |
| **Gly** | 0 | −2 | 0 | −1 | −3 | −2 | −2 | 6 | | | | | | | | | | | | |
| **His** | −2 | 0 | 1 | −1 | −3 | 0 | 0 | −2 | 8 | | | | | | | | | | | |
| **Ile** | −1 | −3 | −3 | −3 | −1 | −3 | −3 | −4 | −3 | 4 | | | | | | | | | | |
| **Leu** | −1 | −2 | −3 | −4 | −1 | −2 | −3 | −4 | −3 | 2 | 4 | | | | | | | | | |
| **Lys** | −1 | 2 | 0 | −1 | −3 | 1 | 1 | −2 | −1 | −3 | −2 | 5 | | | | | | | | |
| **Met** | −1 | −1 | −2 | −3 | −1 | 0 | −2 | −3 | −2 | 1 | 2 | −1 | 5 | | | | | | | |
| **Phe** | −2 | −3 | −3 | −3 | −2 | −3 | −3 | −3 | −1 | 0 | 0 | −3 | 0 | 6 | | | | | | |
| **Pro** | −1 | −2 | −2 | −1 | −3 | −1 | −1 | −2 | −2 | −3 | −3 | −1 | −2 | −4 | 7 | | | | | |
| **Ser** | 1 | −1 | 1 | 0 | −1 | 0 | 0 | 0 | −1 | −2 | −2 | 0 | −1 | −2 | −1 | 4 | | | | |
| **Thr** | 0 | −1 | 0 | −1 | −1 | −1 | −1 | −2 | −2 | −1 | −1 | −1 | −1 | −2 | −1 | 1 | 5 | | | |
| **Trp** | −3 | −3 | −4 | −4 | −2 | −2 | −3 | −2 | −2 | −3 | −2 | −3 | −1 | 1 | −4 | −3 | −2 | 11 | | |
| **Tyr** | −2 | −2 | −2 | −3 | −2 | −1 | −2 | −3 | 2 | −1 | −1 | −2 | −1 | 3 | −3 | −2 | −2 | 2 | 7 | |
| **Val** | 0 | −3 | −3 | −3 | −1 | −2 | −2 | −3 | −3 | 3 | 1 | −2 | 1 | −1 | −2 | −2 | 0 | −3 | −1 | 4 |

# Substitution matrix

Using a substitution matrix which includes positive scores for matches (and quasi matches) allows to simplify the recursive computation of alignment scores!

Needleman-Wunsch algorithm:

- Given sequences $a = a_1 a_2 \dots a_n$ and $b = b_1 b_2 \dots b_m$
- Cells $A(0, j)$ of the first row will have scores $j \times \mathrm{w_d}$ (linear gap penalty)
- Cells $A(i, 0)$ of the first column will have scores $i \times \mathrm{w_d}$ (linear gap penalty)
- Recursive computation:

$$A\big(a(i), b(j)\big) = \mathbf{max} \begin{cases} \mathbf{w_d} + A\big(a(i), b(j-1)\big) \\ \mathbf{w_d} + A\big(a(i-1), b(j)\big) \\ \sigma(\mathbf{a_i}, \mathbf{b_j}) + A\big(a(i-1), b(j-1)\big) \end{cases}$$

  - $\mathbf{w_d} < 0$: negative **gap penalty** for insertions/deletions
  - $\sigma(\mathbf{a_i}, \mathbf{b_j})$: substitution score for $a_i$ and $b_j$ (subtitution matrix)
  - Both, matches and mismatches are included in $\sigma$; no need to distinguish!

# Example

| $\sigma$ | A | C | G | T |
|---|---|---|---|---|
| **A** | **+1** | -1 | -1 | -1 |
| **C** | -1 | **+1** | -1 | -1 |
| **G** | -1 | -1 | **+1** | -1 |
| **T** | -1 | -1 | -1 | **+1** |

- Gap penalty ($w_d$): -2

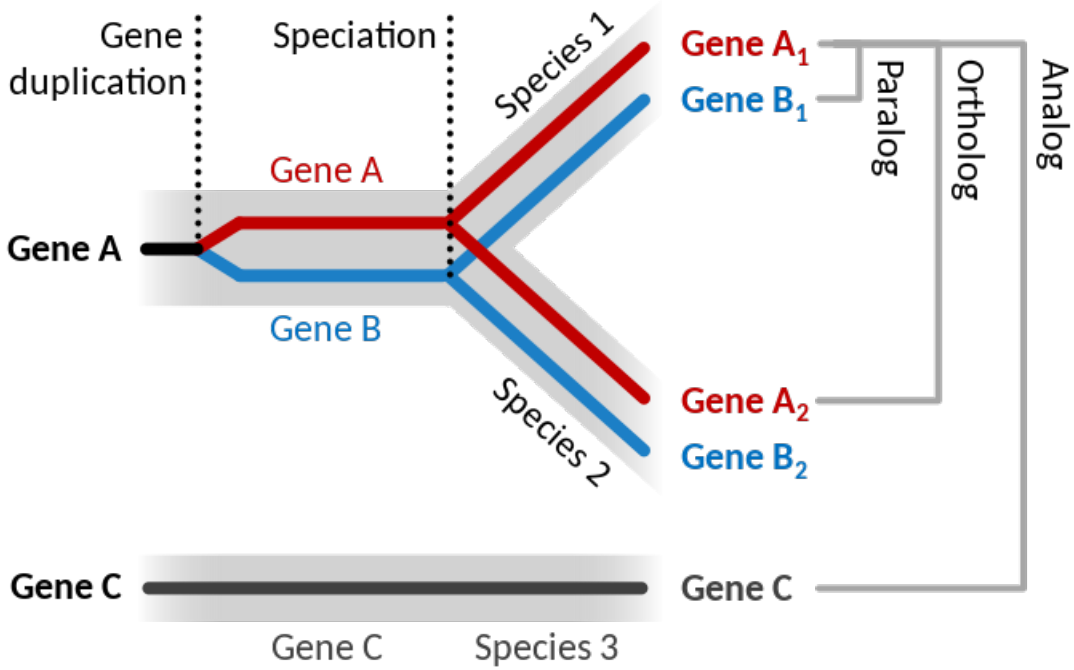
- Sequences to be aligned: "GAAC" and "CAAGAC"

| | - | C | A | A | G | A | C |
|---|---|---|---|---|---|---|---|
| - | 0 | -2 | -4 | -6 | -8 | -10 | -12 |
| G | -2 | -1 | -3 | -5 | -5 | -7 | -9 |
| A | -4 | -3 | 0 | -2 | -4 | -4 | -6 |
| A | -6 | -5 | -2 | 1 | -1 | -3 | -5 |
| C | -8 | -5 | -4 | -1 | 0 | -2 | -2 |

  - Initialize the matrix
  - Cell $(1, 1)$: choice between -2-2 (horizontal), -2-2 (vertical) and 0-1 (diagonal)
  - First row ...
  - Second row ...
  - Full matrix; the final alignment score is -2
  - C A A G A C
    G A A – – C

# Use of global sequence alignment

- Global sequence alignment is the “de facto” standard for comparing biological sequences that are directly related by evolution
  - Homologous genes or proteins between different species
    - ★ The “same” gene inherited from a common ancestor (“orthologous” genes)
    - ★ E.g., *SHH* in human and mouse; *Pax6* in vertebrates and *eyeless* in flies
  - Homologous genes or proteins due to duplication events
    - ★ Genes derived by duplication from a single ancestral gene (“paralogous” genes)
    - ★ E.g., $\alpha$*-haemoglobin* and $\beta$*-haemoglobin* in human (or in mouse)




(from: `https://www.differencebetween.com/difference-between-orthologous-and-paralogous-genes/`)

  - How about analogous sequences (similar function but no evolutionary relationship)?

# Shortcoming of global sequence alignment

- Different types of macromolecules evolve at different speeds
  - Protein-coding genes (DNA) often evolve faster than the encoded proteins
    - ⋆ Redundancy of the genetic code (different codons can encode for the same amino acid)
    - ⋆ Chemically and structurally similar amino acids can often be substituted without effecting protein structure or function
  - Within a genome (and also within individual genes), some regions tend to be more conserved by evolution than others
    - ⋆ Genes are more conserved than the rest of the DNA
    - ⋆ Within genes, exons are more conserved than introns
    - ⋆ Within the coding regions of a gene (and within the protein it encodes) some regions are more conserved (e.g., functionally essential domains; negative selection against sequence alterations)
- Thus, in many cases, an evolutionary relationship can be identified only for a part of the sequences, but not "globally", while for the rest of the sequences often no relevant similarity can be found
- We need a way of identifying similar regions **within** two sequences without requiring the full sequences to be aligned!

# Local sequence alignment

- Let $a = a_1 a_2 \ldots a_n$ and $b = b_1 b_2 \ldots b_m$ be two strings over alphabet $\Sigma$ for which we want to identify consecutive substrings which have a high degree of similarity
- Let $\sigma(s_i, s_j)$ be a substitution matrix with scores for all $s_i, s_j \in \Sigma$
- Let $w_d$ be a gap penalty (we consider only the linear model here; no distinction between opening and extending a gap)

- Let's continue to use our dynamic programming-based approach ...

- **Problem:**

  **Find two substrings, one from $a$ and one from $b$,**
  **that produce the alignment (of the substrings!)**
  **with the maximum possible score**

  or in other words:

  **Find the partial alignment (of substrings of $a$ and $b$),**
  **with the maximum score (such that there are no other**
  **substrings which would achieve a higher alignment score)**

- Of course we cannot know in advance how long the substrings need to be!

# Local sequence alignment

- **Problem:**

  **Find two substrings, one from $a$ and one from $b$,**
  **that produce the alignment (of the substrings!)**
  **with the maximum possible score**

- Observation: if the two sequences are not identical (e.g., if $a \neq b$), there **must** be operations with negative score (substitutions, insertions, deletions)

- Two (rare) special cases:
  - Since negative scores are possible, if there is no pair of substrings with an alignment score $> 0$, then the best solution is trivally given by two empty strings $\varepsilon$ (which have alignment score 0); trivial example: $a$="AAAAA" and $b$="GGGGG"
  - If we have no negative scores in the global alignment (only positive or zero) then, then the best solution would be the full global alignment

- Complexity of the problem:
  - The two substrings with maximum alignment score need not be of the same length
  - Thus, we need a way to compare and align **all** possible substring pairs from $a$ and $b$
  - We have $O(n^2)$ and $O(m^2)$ substrings, respectively
  - Thus, naively enumerating all possible pairs of substrings would require to compute $O(n^2 m^2)$ alignments ...

# The “Smith-Waterman” algorithm

- Introduced in 1981 by Smith and Waterman
- Designed explicitly for the comparison of biological sequences
- Guarantees to find the best (optimal) solution

- $O(nm)$ instead of $O(n^2m^2)$! How?
- It’s based again on the dynamic programming matrix!

- Recall the global sequence alignment:

| | - | C | A | A | G | A | C |
|---|---|---|---|---|---|---|---|
| - | 0 | -2 | -4 | -6 | -8 | -10 | -12 |
| G | -2 | -1 | -3 | -5 | -5 | -7 | -9 |
| A | -4 | -3 | 0 | -2 | -4 | -4 | -6 |
| A | -6 | -5 | -2 | 1 | -1 | -3 | -5 |
| C | -8 | -5 | -4 | -1 | 0 | -2 | -2 |

  - This already *does* contain scores which potentially consider all possible substring pairs (selecting, however, in each step only possibilities which increase the score)!
  - E.g., in the diagonal path from cell $(0,3)$ to cell $(2,5)$ we increase the score from -6 to -4; so the correspondig partial alignment (`GA` and `GA`) would have a score of 2
  - Problem: how can we identify which possible subpath has the highest increase in score?

# The “Smith-Waterman” algorithm

- Recall the global sequence alignment:

| | - | C | A | A | G | A | C |
|---|---|---|---|---|---|---|---|
| - | 0 | -2 | -4 | -6 | -8 | -10 | -12 |
| G | -2 | -1 | -3 | -5 | -5 | -7 | -9 |
| A | -4 | -3 | 0 | -2 | -4 | -4 | -6 |
| A | -6 | -5 | -2 | 1 | -1 | -3 | -5 |
| C | -8 | -5 | -4 | -1 | 0 | -2 | -2 |

  - Problem: how can we identify which possible subpath has the highest increase in score?

- Using global sequence alignment, cell $(i, j)$ contains the total alignment score for the prefixes $a(i)$ and $b(j)$, i.e., from the beginning, represented by cell $(0, 0)$

- Idea: let cell $(i, j)$ instead contain the maximum alignment score of all substring pairs which **end** at position $i$ and $j$, respectively; that is, not from the beginning, but from some intermediate positions in $a$ and $b$

# The “Smith-Waterman” algorithm

- We can achieve this by adding a simple 4th choice for each step:

$$M(i,j) = \mathbf{max} \begin{cases} \mathbf{0} \\ \mathbf{w_d} + M(i,j-1) \\ \mathbf{w_d} + M(i-1,j) \\ \sigma(\mathbf{a_i}, \mathbf{b_j}) + M(i-1,j-1) \end{cases}$$

  - **The additional choice is ... zero!**

  - The motivation for this is the following:
  - Computing a value for a cell (and keeping the corresponding traceback pointer) is equivalent to extending an alignment
  - But: if all possible extensions lead to a negative score, i.e., if $M(i,j)$ would be $< 0$, then it’s better to “reset” everything and start a new potential alignment, i.e., start from 0 again
    - ⋆ Remember: we always can have a score of zero by aligning two empty strings, so any better solution must have a positive score!

  - This implies: when the choice is **0** (resetting), **no traceback pointer** will be associated with the cell (no extension of a previous alignment!)

# The “Smith-Waterman” algorithm: example

| $\sigma$ | A | C | G | T |
|---|---|---|---|---|
| **A** | **+1** | -1 | -1 | -1 |
| **C** | -1 | **+1** | -1 | -1 |
| **G** | -1 | -1 | **+1** | -1 |
| **T** | -1 | -1 | -1 | **+1** |

- Gap penalty ($w_d$): -2

- Sequences to be aligned: “GAAC” and “CAGAAT”

| | - | C | A | G | A | A | T |
|---|---|---|---|---|---|---|---|
| - | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| G | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| A | 0 | 0 | 1 | 0 | 2 | 1 | 0 |
| A | 0 | 0 | 1 | 0 | 1 | 3 | 1 |
| C | 0 | 1 | 0 | 0 | 0 | 1 | 2 |

  - Initialize the matrix:
    - ★ Zeroes in first row and column (instead of negative scores from gaps!)
  - Cell $(1,1)$: choices are 0-2 (horizontal), 0-2 (vertical), 0-1 (diagonal), and 0
    - ★ Remember: no traceback pointer placed for choice 0!
  - Cell $(1,3)$: choices are 0-2 (horizontal), 0-2 (vertical), 0+1 (diagonal), and 0
    - ★ We can increase the score and set a traceback pointer!
  - For the remaining cells of row 1 we always choose 0
  - Row by row ...
  - Placing traceback pointers only when not resetting!

# The “Smith-Waterman” algorithm: example

| $\sigma$ | A | C | G | T |
|---|---|---|---|---|
| **A** | **+1** | -1 | -1 | -1 |
| **C** | -1 | **+1** | -1 | -1 |
| **G** | -1 | -1 | **+1** | -1 |
| **T** | -1 | -1 | -1 | **+1** |

- Gap penalty ($w_d$): -2

- Sequences to be aligned: “GAAC” and “CAGAAT”

| | - | C | A | G | A | A | T |
|---|---|---|---|---|---|---|---|
| - | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| G | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| A | 0 | 0 | 1 | 0 | 2 | 1 | 0 |
| A | 0 | 0 | 1 | 0 | 1 | 3 | 1 |
| C | 0 | 1 | 0 | 0 | 0 | 1 | 2 |

  - Once the table is complete: where is the “optimal score”? Still in cell $(n, m)$?
  - The optimal score is the ***highest positive score*** *in the entire matrix*!
  - This is the alignment score for the best local alignment! Here: 3

  - Traceback to construct the local alignment: start from the cell with the highest score!

```
G A A
G A A
```

★ Traceback proceeds as usual

★ **Until we reach a cell without pointer!**

# The “Smith-Waterman” algorithm: example

| $\sigma$ | A | C | G | T |
|---|---|---|---|---|
| **A** | **+1** | -1 | -1 | -1 |
| **C** | -1 | **+1** | -1 | -1 |
| **G** | -1 | -1 | **+1** | -1 |
| **T** | -1 | -1 | -1 | **+1** |

- Gap penalty ($w_d$): -2

- Sequences to be aligned: “GAAC” and “CAGAAT”

| | - | C | A | G | A | A | T |
|---|---|---|---|---|---|---|---|
| - | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **G** | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| **A** | 0 | 0 | 1 | 0 | 2 | 1 | 0 |
| **A** | 0 | 0 | 1 | 0 | 1 | 3 | 1 |
| **C** | 0 | 1 | 0 | 0 | 0 | 1 | 2 |

  - Traceback to construct the local alignment: start from the cell with the highest score!

```
G A A
G A A
```

  - Symbols associated with the stopping cell with score 0 are not part of the alignment
    - ★ Here: we followed three pointers, so the alignment must have 3 columns!

  - If the traceback goes from cell $(i_s, j_s)$ to cell $(i_e, j_e)$ here: from $(3,5)$ to $(0,2)$
    - ★ $\rightarrow$ The subtring of $a$=GAAC has indices $i_e + 1$ to $i_s$ here: from 1 to 3
    - ★ $\rightarrow$ The subtring of $b$=CAGAAT has indices $j_e + 1$ to $j_s$ here: from 3 to 5

# Additional considerations

- Multiple optimal solutions:
  - Highest score found in multiple cells; or, multiple paths from the same highest score
  - We can:
    - ★ Report only one of them
    - ★ Report them all (best choice!)
  - How about the second best local alignment(s)?
    - ★ Once we've found the best local alignment(s), we can iterate by finding the "second best" score *in cells which haven't been used for the best local alignment* and trace back to find a second best local alignment ...
  - We can iterate the previous point to find the third, fourth, ... best local alignment
    - ★ Remember that we can apply this algorithm to sequences of many hundreds or thousands of nucleotides/amino acids!
    - ★ In biological applications, we indeed often find multiple interesting local alignments between two sequences

# Playing around

- Needleman-Wunsch algorithm, or “global alignment”:
  `http://rna.informatik.uni-freiburg.de/Teaching/index.jsp?toolName=Needleman-Wunsch`

- Smith-Waterman algorithm, or “local alignment”:
  `http://rna.informatik.uni-freiburg.de/Teaching/index.jsp?toolName=Smith-Waterman`

# And so on ...

We've seen a dynamic programming approach for

- Computing the edit distance of two strings
- Finding the LCS of two strings
- Computing the optimal global alignment of two strings, given a substitution matrix and "gap penalty" scores
- Computing the optimal local alignment of two strings, given a substitution matrix and "gap penalty" scores

(You should know how to describe these, e.g., for an exam :-)

Variations of these algorithms:

- The same idea can be applied to solve similar problems, for example:
  - Finding the best local alignment, where the two optimal substrings are one a prefix and the other a suffix, or vice versa
  - Find the best global alignment, where insertions and deletions at the beginning and at the end of the alignment have gap penalty score 0
- You should also be able to design simple "variations" of these alorithms to address slightly different problems to the once we described in the lecture
  - The two examples above are question from previous exams

# References I




Banerjee T.D. et al. (2020)
*Expression of Multiple engrailed Family Genes in Eyespots of Bicyclus anynana Butterflies Does Not Implicate the Duplication Events in the Evolution of This Morphological Novelty*
Front. Ecol. Evol. 8:227



Smith T.F. & Waterman M.S. (1981)
*Identification of Common Molecular Subsequences*
Journal of Molecular Biology 147(1):195–197